

Návrh a implementácia efektívneho komunikačného protokolu pre IoT meteorologické zariadenia

Kontext zadania:

Spoločnosť FIITMeteo vyvíja a nasadzuje meteorologické IoT senzory určené na dlhodobý zber meteorologických údajov. Zariadenia sú napájané z batérie a z tohto dôvodu je efektivita prenášaných dát rozhodujúcim faktorom. Pre tieto zariadenia je nutné pripraviť proprietárny komunikačný protokol, ktorým budú komunikovať zo serverom. Nutné je aj naprogramovať aj samotný server spolu s testovacím programom, ktorý bude emulovať jednotlivé zariadenia.

Príbeh:

Predstavte si, že ste vo firme FIITMeteo na pozícii junior developera. Keďže ste vo firme na tejto pozícii už niečo vyše roka, rozhodol sa vás váš senior developer (váš cvičiaci) preveriť stredne náročnou úlohou – prvý samostatným projektom, pri ktorom budete musieť navrhnuť protokol a implementovať dva programy podľa požiadaviek klienta. Keďže ste takúto úlohu ešte nerobili, dohodol si s vami váš senior developer stretnutie v polovici trvania projektu, kde mu máte predviesť PoC (Proof-of-Concept) vášho riešenia - stačí, ak bude protokol navrhnutý a implementovaný pomocou štruktúr JSON (Prvé odovzdanie zadania). Po tomto stretnutí začnete protokol optimalizovať na binárny, keďže ide o IoT zariadenia napájané batériou, každý ušetrený bajt prenosu predlžuje životnosť zariadenia. Ako to už ale býva, klient niekedy nevie čo presne chce – sedíte v zasadačke spolu s vašim senior developerom a pred tým ako idete projekt odovzdať vám ešte rýchlo zavolá projektový manažér (cvičiaci zodpovedný za zadanie), že zabudol na jednu zmenu, na ktorej sa s klientom dohodli. Táto zmena nie je náročná ani veľká, ale pre klienta dôležitá a keďže máte ešte pol hodinu do stretnutia, rozhodli ste sa túto zmenu zapracovať (doimplementácia). Keď túto zmenu zapracovať zvládnete, prejdete si s klientom najprv akceptačné testy t. j. testy, ktoré musí vaše riešenie splniť, aby mohlo byť prebraté klientom. Potom odprezentujete projekt klientovi (finálne preberanie zadania cvičiacim).

Pred tým ako vám bol tento projekt zadaný, projektový manažér pripravil dokument na základe stretnutí s klientom, podľa ktorého budete musieť projekt vypracovať. V dokumente sú uvedené aj Akceptačné testy (minimálne požiadavky zadania). Zároveň je tam aj vypracovaný systém ohodnotenia projektu, ktorý bol s klientom dohodnutý (bodovanie). Váš senior developer je voči vám transparentný, ak nezískate pre firmu aspoň polovicu ohodnotenia (získanie polovice bodov pri prvom odovzdaní a získanie polovice bodov pri finálnom odovzdaní), nebude vás môcť odporučiť na vyššiu pracovnú pozíciu (pripustenie ku skúške).

Zápis zo stretnutia s klientom:

(Požiadavky na zadanie):

Klient si vybral nasledujúce zariadenia pre svoj projekt:

1. **ThermoNode** (Teplota, vlhkosť a tlak) – 4 parametre
 - **temperature**: teplota v °C (-50.0 až +60.0, desatinné číslo – 1 desatinné miesto)
 - **humidity**: relatívna vlhkosť v % (0.0 – 100.0, desatinné číslo – 1 desatinné miesto)
 - **dew_point**: rosný bod v °C (-50.0 až +60.0, desatinné číslo – 1 desatinné miesto)
 - **pressure**: atmosférický tlak v hPa (800.00 – 1100.00 - , desatinné číslo – 2 desatinné miesta)
2. **WindSense** (Rýchlosť a charakter vetra) – 4 parametre
 - **wind_speed**: rýchlosť vetra v m/s (0.0 – 50.0, desatinné číslo – 1 desatinné miesto)
 - **wind_gust**: náraz vetra v m/s (0.0 – 70.0, desatinné číslo – 1 desatinné miesto)
 - **wind_direction**: smer vetra v stupňoch (0 – 359, celočíselné)
 - **turbulence**: index turbulencie (0.0 – 1.0, desatinné číslo – 1 desatinné miesto)
3. **RainDetect** (Zrážky a pôda) – 4 parametre
 - **rainfall**: úhrn zrážok za poslednú hodinu v mm (0.0 – 500.0, desatinné číslo – 1 desatinné miesto)
 - **soil_moisture**: vlhkosť pôdy v % (0.0 – 100.0, desatinné číslo – 1 desatinné miesto)
 - **flood_risk**: úroveň rizika povodne (0 – none, 1 – low, 2 – medium, 3 – high, 4 hodnoty)
 - **rain_duration**: trvanie zrážok v minútach za poslednú hodinu (0 – 60, celočíselné)
4. **AirQuality** (Kvalita ovzdušia) – 3 parametre
 - **co2**: koncentrácia CO₂ v ppm (300 – 5000, celočíselné)
 - **ozone**: koncentrácia ozónu v µg/m³ (0.0 – 500.0, desatinné číslo – 1 desatinné miesto)
 - **air_quality_index**: AQI (0 – 500, celočíselné)

Požiadavky:

- P1. Navrhnite a implementujte vlastný aplikačný protokol nad UDP.
- P2. Implementujte server, ktorý spracúva a zobrazuje údaje, a jeden testovací program emulujúci všetky typy senzorov. Pri UAT musia programy bežať na dvoch fyzických zariadeniach.
- P3. Každý senzor sa pred odosielaním dát zaregistruje. Server mu prideli token, ktorý senzor používa v ďalších správach danej relácie. Správy s neplatným tokenom sú zamietnuté.

- P4. Testovací program po registrácii automaticky odosiela údaje každého senzora každých 10 sekúnd. Odosielanie možno vypnúť a znova zapnúť. Táto funkcionality je po registrácii automaticky zapnutá.
- P5. Oba programy implementujte v Pythone, iný jazyk je možný po dohode s cvičiacim. UDP komunikáciu implementujte priamo pomocou knižnice socket vrátane vytvorenia a naviazania socketu. Hotové komunikačné a protokolové riešenia nie sú povolené.
- P6. Odosielané dáta budú mať zabezpečené overenie integrity na serveri, ak sú poškodené server si ich znovu vypýta.
- P7. Program bude obsahovať možnosť vnieść chybu do prenášaných dát na strane testera pre akýkoľvek vybraný senzor (nepoše osobitnú správu ale poškodí ďalšiu v poradí).
- P8. Program senzorov bude obsahovať možnosť odoslať dodatočnú dátovú správu s custom údajmi zadanými používateľom pre akýkoľvek vybraný senzor.
- P9. Ak server 15 sekúnd neprijme dáta od senzora s platnou reláciou, vypíše do logov `SENSOR IDLE` správu a odošle prvú `KEEPLIVE` správu, pričom ďalšie posiela každých 5 sekúnd. Senzor s vypnutým odosielaním dát na `KEEPLIVE` naďalej odpovedá. Po 3 správach `KEEPLIVE` bez odpovede (senzor neaktívny) server zruší reláciu, zneplatní token a vypíše `SENSOR DISCONNECTED`. Ak sa senzor odpojí, trvá presne $15s + 5s + 5s + 5s = 30s$, kým server zneplatní reláciu (Po odoslaní 3. `KEEPLIVE` správy server tiež čaká 5s). Senzor má tiež časovač 30s, ktorý sa počíta od poslednej prijatej `KEEPLIVE` správy, ktorý keď uplynie tak ukončí reláciu.
- P10. Program musí byť schopný obslúžiť aj viacero senzorov rovnakého typu, čo znamená, že každý senzor musí mať svoj typ a aj svoj jednoznačný identifikátor, aby sa zabránilo duplicitám.
- P11. Ak testovací program nedostane do 3 sekúnd od odoslania dátovej správy potvrdenie (ACK), odošle tú istú správu znova. Celkovo ju odošle najviac trikrát, raz pôvodne a dvakrát opakovane. Ak ACK nepríde ani do 3 sekúnd od tretieho odoslania, správu zahodí.
- P12. Formát výpisov (logovanie) bude jednotný podľa normy opísanej nižšie a bude v anglickom jazyku.
- P13. IP adresy a komunikačné porty bude možné konfigurovať priamo z rozhrania programu. IP adresy budú validované a bude ošetrovaná chyba ak je port obsadený.
- P14. Program bude robustný a dokáže spracovávať očakávané chyby aby sa predišlo pádu programu(obsadený port, prerušené spojenie, nesprávna vstupná hodnota a pod.)
- P15. Program bude štandardne svoje výpisy (logy) posielať vždy do súboru a bude obsahovať možnosť nastaviť výpis aj do konzoly na účely kontroly.

Zoznam odporúčaných typov správ na splnenie uvedených požiadaviek:

Nasledujúce správy majú odporúčací charakter a zoznam nemusí obsahovať všetky potrebné správy:

1. **Registračná správa** – Senzor oznámi svoj typ a ID a požiada o vytvorenie relácie. Server mu po úspešnej registrácii prideli token.
2. **Potvrdzujúca správa** – Potvrdzuje prijatie alebo spracovanie správy. Možno použiť jednu spoločnú ACK správu alebo osobitné ACK správy pre rôzne situácie.
3. **Odmietajúca správa** – Oznamuje, že správa nebola prijatá, napríklad pre neplatný token alebo chybu integrity.
4. **Správa s dátami** – Prenáša namerané hodnoty senzora.
5. **Vyžiadanie retransmisie** – Vyžiadanie retransmisie poškodennej správy.
6. **Keepalive správa** – Správa na udržanie spojenia.

Formát informačných správ (logov), ktorý je vyžadovaný.

Logy sú v angličtine a používajú jednotný formát:

<TIMESTAMP>		<TYPE>		<LEVEL>		<SOURCE>		<EVENT/MESSAGE>
-------------	--	--------	--	---------	--	----------	--	-----------------

TIMESTAMP má pevný formát YYYY-MM-DDTHH:mm:ss.sssZ. Čas je v UTC a obsahuje milisekundy.

SOURCE jednoznačne označuje pôvodcu správy alebo udalosti. Pri serveri stačí napríklad **SERVER**. Pri testovacom programe sa pri udalosti konkrétneho senzora uvedie aj jeho typ a ID, napríklad **TESTER:ThermoNode:T-01**.

TYPE je určuje jeden z dvoch vopred definovaných druhov logového záznamu:

- **SYSTEM** – lokálna udalosť programu, napríklad spustenie servera, chyba alebo zmena stavu senzora (**SENSOR IDLE**, **SENSOR DISCONNECTED**), okrem povinných správ je ich obsah ľubovoľný.
- **MESSAGE** – odoslanie alebo prijatie správy vlastného protokolu, či už ide o registráciu, dáta, ACK alebo **KEEPALIVE**. V poli **EVENT/MESSAGE** bude uvedený celý obsah správy v čitateľnej podobe.

Označenia **LEVEL** si riešitelia zvolia sami, odporúčajú sa úrovne závažnosti(Severity) Syslogu podľa RFC 5424. Formát RFC nie je vyžadovaný a slúži na inšpiráciu.

EVENT/MESSAGE obsahuje stručný a jasný čitateľný opis udalosti alebo prijaté dáta. Odoslané aj prijaté dátové správy sa logujú **so všetkými hodnotami senzora**. Hodnoty sa vypíšu prehľadne ako objekt s názvami parametrov malými písmenami podľa detailu senzorov uvedeného na začiatku dokumentu, napríklad `{"temperature":21.3,"humidity":55.4,...}`. Ide o podobu *logu*, finálny prenosový protokol zostáva binárny.

```

2026-05-04T13:37:42.000Z | SYSTEM | INFO | SERVER | Server listening on 0.0.0.0:9000

2026-05-04T13:37:42.100Z | MESSAGE | INFO | TESTER:ThermoNode:<ID_SENZORA> |
Sent
{"type":"REGISTER","seq":"<SEQ_REGISTRACIE>","sensor_type":"ThermoNode","sensor_id
":"<ID_SENZORA>"}

2026-05-04T13:37:42.120Z | MESSAGE | INFO | SERVER | Sent
{"type":"ACK","seq":"<SEQ_ACK>","ack_for":"<SEQ_REGISTRACIE>","token":"<PRIDELENY_
TOKEN>"}

2026-05-04T13:37:52.010Z | MESSAGE | INFO | TESTER:ThermoNode:<ID_SENZORA> |
Received by SERVER
{"type":"DATA","seq":"<SEQ_DAT_1>","sensor_id":"<ID_SENZORA>","token":"<PRIDELENY_
TOKEN>","temperature":21.3,"humidity":55.4,"dew_point":11.9,"pressure":1013.25,"crc
":"<CRC_DAT_1>"}

2026-05-04T13:37:54.010Z | MESSAGE | INFO | TESTER:ThermoNode:<ID_SENZORA> |
Received by SERVER
{"type":"DATA","seq":"<SEQ_DAT_2>","sensor_id":"<ID_SENZORA>","token":"<PRIDELENY_
TOKEN>","temperature":23.4,"humidity":48.0,"dew_point":11.7,"pressure":1010.42,"crc
":"<CRC_DAT_2>"}

2026-05-04T13:37:55.000Z | SYSTEM | WARNING | SERVER | Integrity check failed for
ThermoNode:<ID_SENZORA> retransmission requested

2026-05-04T13:38:09.010Z | SYSTEM | INFO | SERVER | SENSOR IDLE:
ThermoNode:<ID_SENZORA>

2026-05-04T13:38:14.010Z | MESSAGE | INFO | SERVER | Sent
{"type":"KEEPALIVE","seq":"<SEQ_KEEPALIVE>","sensor_id":"<ID_SENZORA>","token":"<P
RIDELENY_TOKEN>"}

```

Správy majú informačný charakter a okrem povinných častí a formátu sú plne prispôsobiteľné.

Akceptačné testy (UAT):

Podmienky testovania: komunikácia musí prebiehať medzi dvomi fyzickými zariadeniami, musí byť sledovaná pomocou programu Wireshark.

UAT 1: Pripojenie zariadení na server a automatické generovanie správ.	
Vstupné podmienky	Server: Nachádza sa v hlavnom menu. Testovací program: Nachádza sa v hlavnom menu, senzory nie sú zaregistrované.
Scenár testu	1. Server: Používateľ nastaví IP adresu a port a spustí počúvanie. 2. Testovací program: Používateľ nastaví IP adresu a port servera.

	3. Testovací program: Použivatel zaregistruje 4 senzory. Po registrácii sa odosielanie dát spustí automaticky.
Výstupné podmienky	Server zalogue registráciu všetkých štyroch typov senzorov a každému prideli token. Od každého senzora prijme a zalogue aspoň dve dátové správy s odstupom približne 10 sekúnd. Registrácia a prenos dát sú viditeľné vo Wiresharku.
Výsledok testu	SPLNENÉ/AKCEPTOVATEĽNÉ/NESPLNENÉ

UAT 2: Odoslanie dodatočnej správy používateľom.

Vstupné podmienky	Server: Prijíma dáta od registrovaných senzorov a nachádza sa v menu. Testovací program: Nachádza sa v hlavnom menu, automatické odosielanie dát je zapnuté.
Scenár testu	1. Testovací program: Použivatel vyberie možnosť zadať vlastné hodnoty a zvolí ľubovoľný registrovaný senzor. 2. Testovací program: Použivatel zadá hodnoty všetkých parametrov vybraného senzora v povolených rozsahoch. 3. Testovací program: Odošle dátovú správu so zadanými hodnotami.
Výstupné podmienky	Server prijme dodatočnú dátovú správu a zalogue presne tie hodnoty, ktoré používateľ zadal. Automatické odosielanie dát ostatných senzorov pokračuje. Správa je viditeľná vo Wiresharku.
Výsledok testu	SPLNENÉ/AKCEPTOVATEĽNÉ/NESPLNENÉ

UAT 3: Zavedenie chyby do odosielaných dát.

Vstupné podmienky	Server: Prijíma dáta od registrovaných senzorov a nachádza sa v hlavnom menu. Testovací program: Nachádza sa v hlavnom menu, automatické odosielanie dát je zapnuté.
Scenár testu	1. Testovací program: Použivatel vyberie možnosť zaviesť chybu a zvolí jeden senzor. 2. Testovací program: Poškodí nasledujúcu dátovú správu vybraného senzora tak, aby neprešla kontrolou integrity. Neodosiela osobitnú správu na oznámenie chyby. 3. Server: Zalogue chybu integrity a vyžiada opätovné odoslanie poškodenej správy. 4. Testovací program: Znovu odošle tú istú dátovú správu bez poškodenia.
Výstupné podmienky	Server poškodenú správu neprijme ako platné meranie. Po retransmisii zalogue správne prijaté hodnoty. Poškodená správa, vyžiadanie retransmisie a správne opakovanie sú viditeľné vo Wiresharku.
Výsledok testu	SPLNENÉ/AKCEPTOVATEĽNÉ/NESPLNENÉ

UAT 4: Odpojenie zariadenia a mechanizmus kontroly aktivity zariadenia	
Vstupné podmienky	Server: Prijíma dáta od registrovaných senzorov a nachádza sa v hlavnom menu. Testovací program: Nachádza sa v hlavnom menu, automatické odosielanie dát je zapnuté.
Scenár testu	1. Testovací program: Používateľ vyberie jeden zo senzorov a vypne mu odosielanie dát. Senzor naďalej odpovedá na KEEPALIVE a udržiava platnú reláciu. 2. Server: Po 15 sekundách bez dát zaloguje SENSOR IDLE a začne overovať dostupnosť senzora. 3. Testovací program: Používateľ znovu zapne odosielanie dát a počká na prijatie jednej dátovej správy serverom. 4. Testovací program: Používateľ ukončí lokálnu reláciu vybraného senzora(môže byť testované aj fyzickým prerušením spojenia t.j. vytiahnutím kábla). Tento senzor prestane posielať dáta aj odpovede na KEEPALIVE. 5. Server: Po 3 nezodpovedaných správach KEEPALIVE zruší reláciu vybraného senzora a zneplatní jeho token. 6. Testovací program: Používateľ znovu zaregistruje odpojený senzor.
Výstupné podmienky	Pri vypnutí iba dát server zaloguje SENSOR IDLE, ale reláciu nezruší. Po úplnom odpojení zaloguje SENSOR DISCONNECTED a zneplatní pôvodný token. Pri opätovnej registrácii prideli senzoru nový token a znovu prijíma jeho dáta. Priebeh KEEPALIVE a registrácie je viditeľný vo Wiresharku.
Výsledok testu	SPLNENÉ/AKCEPTOVATEĽNÉ/NESPLNENÉ

UAT 5: Znovu odoslanie dát, ak server nepotvrdí prijatie správy.	
Vstupné podmienky	Server: Prijíma dáta od registrovaných senzorov a nachádza sa v hlavnom menu. Testovací program: Nachádza sa v hlavnom menu, automatické odosielanie dát je zapnuté.
Scenár testu	1. Server: Používateľ vyberie senzor, ktorému server nepotvrdí prijatie x (ľubovoľný počet) dátových správ(aj opakovaných). 2. Testovací program: Odošle dátovú správu. Keď do 3 sekúnd nedostane ACK, odošle tú istú správu znova a čaká znovu 3 sekundy. 3. Testovací program: Ak znovu ACK nepríde, po ďalších 3 sekundách odošle správu tretí krát(znovu počká 3 sekundy a potom ju zahodí). 4. Server: Po nepotvrdení x správ server znovu začne odpovedať. (Ak by bolo $x=5$ tak sa pošle 3 krát správa č.1 bez odpovede, následne 2 krát správa č.2 bez odpovede ale na tretí krát sa už

	správa č.2 serverom prijme. Teda 5 pokusov o odoslanie bolo potlačených, bez ACK, 6ty pokus úspešne prijatý.)
Výstupné podmienky	Vo Wiresharku je viditeľných x odoslání dátových správ bez odpovede a následné ACK.
Výsledok testu	SPLNENÉ/AKCEPTOVATEĽNÉ/NESPLNENÉ

UAT4 a UAT5 môžu byť testované aj fyzickým odpojením kábla z portu zariadenia, preto netreba mapovať konkrétne správanie iba na výber z menu. Protokol by sa mal správať vždy deterministicky bez ohľadu na to, čo je zvolené na testovanie na jednej alebo druhej strane.

Proof of Concept (prvé odovzdanie):

Návrh a implementácia JSON protokolu - formát JSON v čitateľnej forme na zjednodušenie odladenia a overenie funkčnosti jednotlivých programov ako aj toho ako máte protokol navrhnutý na úrovni správ.

Hodnotenie:

Všetky položky označené červeným sú povinné aspoň v akceptovateľnej podobe. Zároveň však platí, že je nutné získať minimum 2,5 bodu v súčte. Je to pre to, aby sme mohli prebrať aj zadanie, ktoré má menšie nedostatky.

Čo sa hodnotí	Nesplnené	Akceptovateľné	Splnené/Maximum
UAT 1	0 b	0,5 b	1 b
UAT 2	0 b	0,5 b	1 b
UAT 3-5	0 b	0,5 b	1 b
UAT 3-5	0 b	0,5 b	1 b
Dokumentácia	0 b	0,5 b	1 b
SPOLU		Min 2,5b	5 b

Dokumentácia musí v tomto bode obsahovať minimálne tieto náležitosti. Zároveň musí byť odovzdaná ako PDF podľa vzoru, ktorý máte k dispozícii:

- Vývojové diagramy s opisom cyklu prijímania a cyklu odosielania v oboch programoch. Krátky slovný popis.
- Návrh jednotlivých správ v JSON aj s príkladom pre každú správu.
- Sekvenčný diagram pre komunikáciu pre UAT1 a UAT2 (zvoľte vhodnú úroveň abstrakcie pre zachovanie čitateľnosti a prehľadnosti).
- Zoznam použitých knižníc.
- Návrh merania efektivity prenášaných dát (nezabudnúť počítať aj s nižšími vrstvami), ako bude prebiehať meranie aby ste jasne ukázali rozdiel medzi JSON a binárnym variantom protokolu.
- Diagramy a obrázky musia byť jasne čitateľné.

Výstupy:

Súbor **.zip** (iné kompresie (.rar, .7z, .gz..) **nebudú akceptované!**) pomenovaný xpriezvisko.zip obsahujúci:

- Program servera pomenovaný server.xxx
- Testovací program pomenovaný tester.xxxx
- PDF súbor dokumentácie pomenovaný xpriezvisko.pdf

Optimalizácia na binárny protokol (finálne odovzdanie):

Optimalizovaný protokol, ktorého funkčnosť je overená dvomi implementovanými programami. Cieľom je reprezentovať typ správy efektívne (napr. 2 bity pre 4 typy zariadení alebo napr. Timestamp zakódovať ako 32-bitové číslo, ak to stačí). Použite čo najmenší možný počet bitov pre každý parameter (napr. ak parameter má obmedzený rozsah a presnosť, zvolí sa vhodná reprezentácia), rovnako treba využívať aj príznaky (jednobitové parametre). Pripravte dissector pre wireshark vášho protokolu v jazyku LUA.

Hodnotenie:

Všetky položky označené červeným sú povinné aspoň v akceptovateľnej podobe. Zároveň však platí, že je nutné získať minimum 7,5 bodov v súčte. Je to pre to, aby sme mohli prebrať aj zadanie, ktoré má menšie nedostatky.

Čo sa hodnotí	Nesplnené	Akceptovateľné	Splnené/Maximum
UAT 1	0 b	1 b	1 b
UAT 2	0 b	1 b	1 b
UAT 3	0 b	1 b	1,5 b
UAT 4	0 b	1 b	1,5 b
UAT 5	0 b	1 b	1,5 b
LUA script	0 b	0,5 – 1b – Podľa rozsahu prípadov, ktoré skript pokrýva.	
Meranie efektivity protokolu		0,5 – 1 b – Prehľadné, opakovateľné porovnanie vlastných protokolov, podporené tabuľkou a grafom.	1,5 – 2 b – Dôkladné porovnanie vlastného protokolu s existujúcim protokolom používaným v praxi, podporené tabuľkou a grafom.
Efektivita prenášaných údajov	0 b – Použitie veľkých dátových typov na malé čísla a pod.	0,5 – 1,5 b – Vhodné použitie dátových typov, avšak efektivita by sa dala vylepšiť.	2 b – Vysoko efektívny protokol, meniace sa dĺžky podľa potreby a pod.
Dokumentácia		0,5 b – obsahuje všetky náležitosti	2 b – Prehľadná organizovaná dokumentácia bez vaty, podľa ktorej by bolo možné tento protokol naprogramovať bez zbytočných otázok.

Pochopenie problematiky v širšom kontexte.	0 – 0,5b – Celkové zhodnotenie projektu, protokolu a samotných výsledkov meraní a porovnávaní.		
Doimplementácia		0,5 b	1 b
SPOLU		Min. 7,5 b	15 b

Dokumentácia musí obsahovať všetky náležitosti. Zároveň musí byť odovzdaná ako PDF podľa vzoru, ktorý máte k dispozícii:

- Vývojové diagramy s opisom cyklu prijímania a cyklu odosielania v oboch programoch. Krátky slovný popis.
- Návrh jednotlivých správ v JSON aj s príkladom pre každú správu.
- Štruktúry binárnych hlavičiek a polí (ideálne v podobnom tvare ako cheatsheet na analýzu rámcov).
- Sekvenčný diagram pre komunikáciu pre UAT1-5.
- Zoznam použitých knižníc.
- Príklad zapúzdrenia správy až na úroveň L2
- Meranie efektivity prenášaných dát (nezabudnúť počítať aj s nižšími vrstvami.
- Porovnanie efektivity prenášaných dát medzi vlastnými protokolmi
- Príloha A: (okolo 150 slov) – Zhodnotenie edukatívnej stránky projektu: Čo som sa vďaka tomuto zadaniu naučil. Ako my môžeme tieto znalosti pomôcť v ďalšom štúdiu a na trhu práce.

Výstupy:

Súbor **.zip** (iné kompresie (**.rar**, **.7z**, **.gz**..) **nebudú akceptované!**) pomenovaný xpriezvisko.zip obsahujúci:

- Program servera pomenovaný server.xxx
- Testovací program pomenovaný tester.xxxx
- PDF súbor dokumentácie pomenovaný xpriezvisko.pdf
- Priečinok s LUA súbormi.

Odporúčanie pre dokumentáciu: Predstavte si, že si kúpite produkt (klúčne nejaký program/softvér) a chcete porozumieť ako funguje. Dostanete do rúk vašu dokumentáciu. Boli by ste spokojný zákazník? Nájdete tam len podstatné veci bez zbytočnej vaty? Je to jasné, čitateľné a prehľadné? Dá sa jednoducho zorientovať? Viete jednoducho pochopiť architektúru a fungovanie? Je možné jednoducho pochopiť obsah diagramov a obrázkov tak, aby čitateľ nemusel brať lupu a poznámkový blok? Vedeli by ste naprogramovať nástroj podľa tejto dokumentácie?

Odporúčanie pre meranie efektivity: Ak chcete vhodne porovnať protokoly, potrebujete rovnaké vstupné podmienky a rovnaký scenár. Koľko percent z vášho protokolu tvorí manažment (typ správ, checksum, sekvenčné čísla,...) a koľko samotné dáta? Neporovnávam v mojom meraní neporovnateľné? Zamyslel som sa či mi čísla ktoré vidím dávajú zmysel? Keď si niekto prečíta toto porovnanie pochopí na prvý pohľad o čo ide? alebo bude musieť zvolať konzílium odborníkov na dešifrovanie najpravdepodobnejšieho zámeru autora?

Všeobecné odporúčanie: Predstavte si, že ste zákazník ktorý si objednal vytvorenie projektu nového protokolu. Idete si skontrolovať či je všetko splnené, budete spokojný s jednotlivými časťami a s fungovaním? Bude vaše riešenie použiteľné v produkcii? Prijali by ste takýto produkt a zaplatili dodávateľovi alebo by ste mu to dali prepracovať?

Používanie AI:

Ak pri doimplementácii bude vaše IDE (vývojové prostredie – PyCharm, VS Code, vim,..) akokoľvek prepojené s AI pluginmi, modulmi a pod. Bude zadanie automaticky hodnotené známkou FX.

Všetky časti pre ktoré sa rozhodnete využiť prostriedky umelej inteligencie, bude treba jasne označiť. Študenti budú musieť taktiež preukázať, že zadanie programovali, tým spôsobom, že budú bez väčšieho zaváhania vedieť vysvetliť ľubovoľnú časť kódu ktorý odovzdajú.